

Arquitetura e Organização de Computadores

Estrutura e Função do Processador

Rodrigo Calvo
rcalvo@uem.br

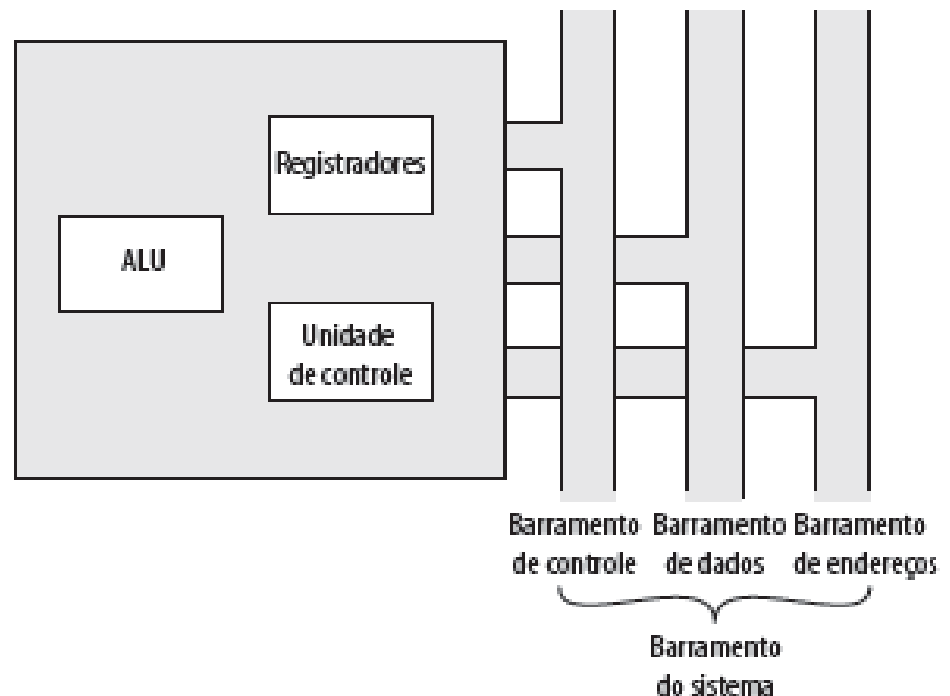
Departamento de Informática – DIN
Universidade Estadual de Maringá – UEM

Estrutura da CPU

- Ações que a CPU deve executar:
 - Buscar instruções.
 - Interpretar instruções.
 - Obter dados.
 - Processar dados.
 - Gravar dados.

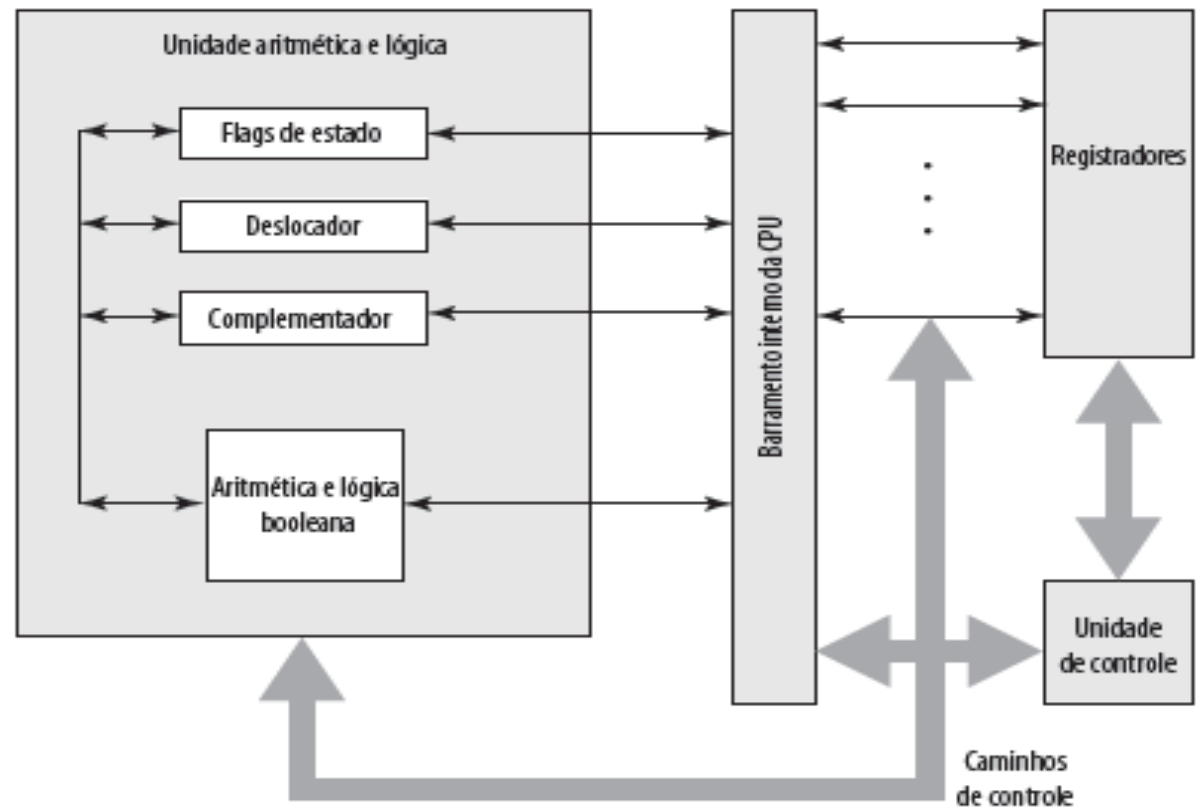
CPU e o Barramento do Sistema

- Barramento do sistema
- ALU faz o processamento do dados.
- UC controla não só a transferência de dados e instruções para dentro e fora da CPU, mas também a própria operação da ALU.
- Registradores: memória de trabalho.



Estrutura Interna da CPU

- Barramento interno
- CPU: unidade de controle, ULA e registradores.
- Computador: CPU, I/O e memória.



Registradores

- CPU precisa ter algum espaço de trabalho (armazenamento temporário).
- Espaço de trabalho = Registradores.
- Número e função variam entre projetos de processador.
- Uma das principais decisões de projeto.
- Nível mais alto da hierarquia de memória.

Organização dos Registradores

Registradores visíveis ao usuário

- Possibilitam que o programador de linguagem de máquina (Assembly) minimize as referências à memória, pela otimização do uso de registradores.

Registradores de controle e estado

- Usados pela unidade de controle para controlar a operação do processador e por programas privilegiados do Sistema Operacional para controlar a execução de programas.

Registradores visíveis ao usuário

- Podem ser referenciados pela linguagem de máquina do computador (Assembly).
- Subdivididos em:
 - Uso geral: Podem ser para uma variedade de funções.
 - Dados: Usados apenas para guardar dados.
 - Endereços: Podem ser usados para um modo de endereçamento em particular (registradores de índice, ponteiros de pilha).
 - Códigos condicionais: Bits definidos pelo hardware do processador como resultado das operações. Ex: operações aritméticas podem resultar resultados positivos, negativos, zero ou *overflow*.

Registradores visíveis ao usuário

- Podem ser de propósito geral verdadeiro.
 - Ortogonais ao código de operação - Qualquer registrador pode conter um operando para qualquer código de operação.
 - Podem ser restritos.
 - Registradores dedicados para operações sobre ponto flutuante ou operações sobre a pilha.
- Podem ser dados
 - Registradores que guardam somente dados

Registradores visíveis ao usuário

- Podem ser usados para cálculo de endereçamento.
 - Registradores de segmento - Contém o endereço base de um segmento de memória.
 - Registradores de índice - Utilizados para endereçamento indexado;
 - Apontador de topo de pilha - Aponta o topo da pilha.

Registradores de uso geral:

Decisão de Projeto

- Torne-os de uso geral:
 - Aumenta a flexibilidade e opções do programador.
 - Aumenta o tamanho de instrução e complexidade.
- Torne-os especializados:
 - Permite referência implícita aos registradores.
 - Instruções menores e mais rápidas.
 - Menor flexibilidade na programação.
- A tendência tem sido utilizar registradores especializados.

Quantos registradores de uso geral?

- O número de registradores afeta o projeto do conjunto de instruções: Quanto mais registradores, maior o número de bits para especificar os operandos.
- Número aceito: entre 8 –32.
- Menos = mais referências à memória.
- Mas muitos não reduz necessariamente as referências à memória e ocupa espaço no processador.
- Algumas arquiteturas RISC podem ter centenas de registradores.

De qual tamanho?

- Grande o suficiente para conter o maior endereço do sistema.
- Grande o suficiente para conter uma palavra completa.
- É desejável que se possa combinar dois registradores de dados para conter valores de tamanho duplo.
 - Na linguagem C podemos fazer:

double a;

long a;

Registradores de código condicional

- Conjuntos de bits individuais (*flags*), atualizados pelo *hardware* da CPU como resultado de operações:
 - Exemplo: Se o resultado da última operação foi zero.
- Podem ser lidos (implicitamente) por programas.
 - Exemplo: Jump if zero (JZ ou BRZ).
- Não podem (normalmente) ser setados por programas.

Registradores de controle de estado

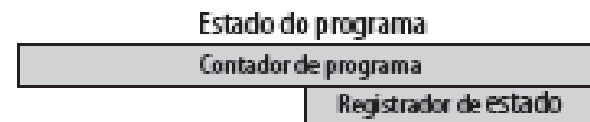
- São utilizados pela UC para controlar a operação da CPU e por programas do SO para controlar a execução de programas.
- Não são visíveis ao usuário na maioria das máquinas.
- Program Counter (PC): O endereço da instrução a ser buscada.
- Instruction Register (IR): A última instrução buscada.
- Memory Address Register (MAR): O endereço de uma posição de memória.
- Memory Buffer Register (MBR): Um dado a ser escrito na memória ou lido recentemente.

Outros registradores

- Dependendo da CPU, podem existir outros registradores de uso específico:
 - Blocos de controle de processo (visto em SO);
 - Vetores de interrupção (visto em SO);
 - Topo de pilha;
 - Controle de operações de I/O.
- No projeto da organização dos registradores, as necessidades do SO são determinantes.

Exemplo de organização de registradores

- MC68000 (registradores separados, mas de uso misto: flexibilidade).
 - Endereços e dados separados.
 - Dados de 8, 16 ou 32 bits.
 - A7 é utilizado como apontador de pilha.



(a) MC68000

Exemplo de organização de registradores

- 8086 (Registradores de uso especial, poucas exceções: Menor flexibilidade)
 - Dados de 16 ou 8 bits.
 - Apontadores e índices de 16 bits.
- 80386: Usa registradores de 32 bits que retém a organização do 8086, para compatibilidade.

Registradores gerais

AX	Acumulador
BX	Base
CX	Contador
DX	Dados

Ponteiros e índices

SP	Ponteiro de pilha
BP	Ponteiro de base
SI	Índice de origem
DI	Índice de destino

Segmento

CS	Código
DS	Dados
SS	Pilha
ES	Extra

Estado de programa

Flags
Ponteiro de instrução

(b) 8086

Registradores gerais

EAX	AX
EBX	BX
ECX	CX
EDX	DX

ESP

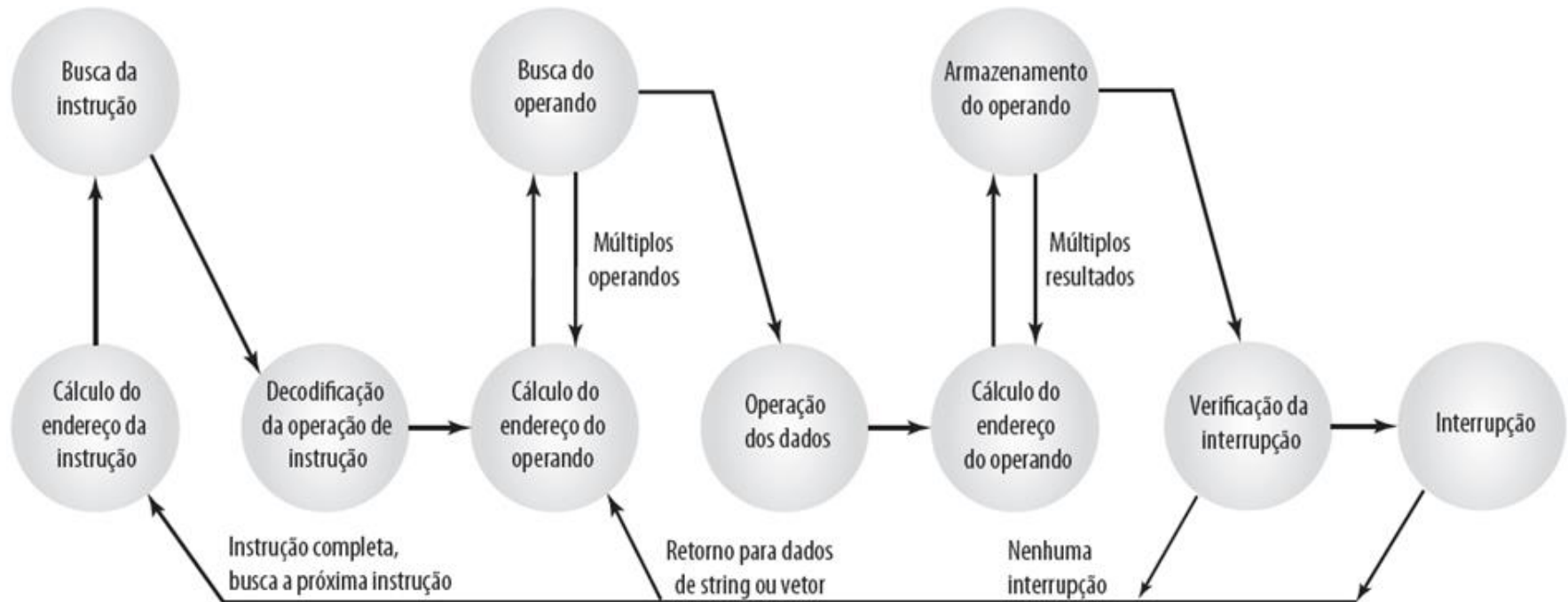
ESP	SP
EBP	BP
ESI	SI
EDI	DI

Estado do programa

Registrador FLAGS
Ponteiro da instrução

(c) 80386 — Pentium 4

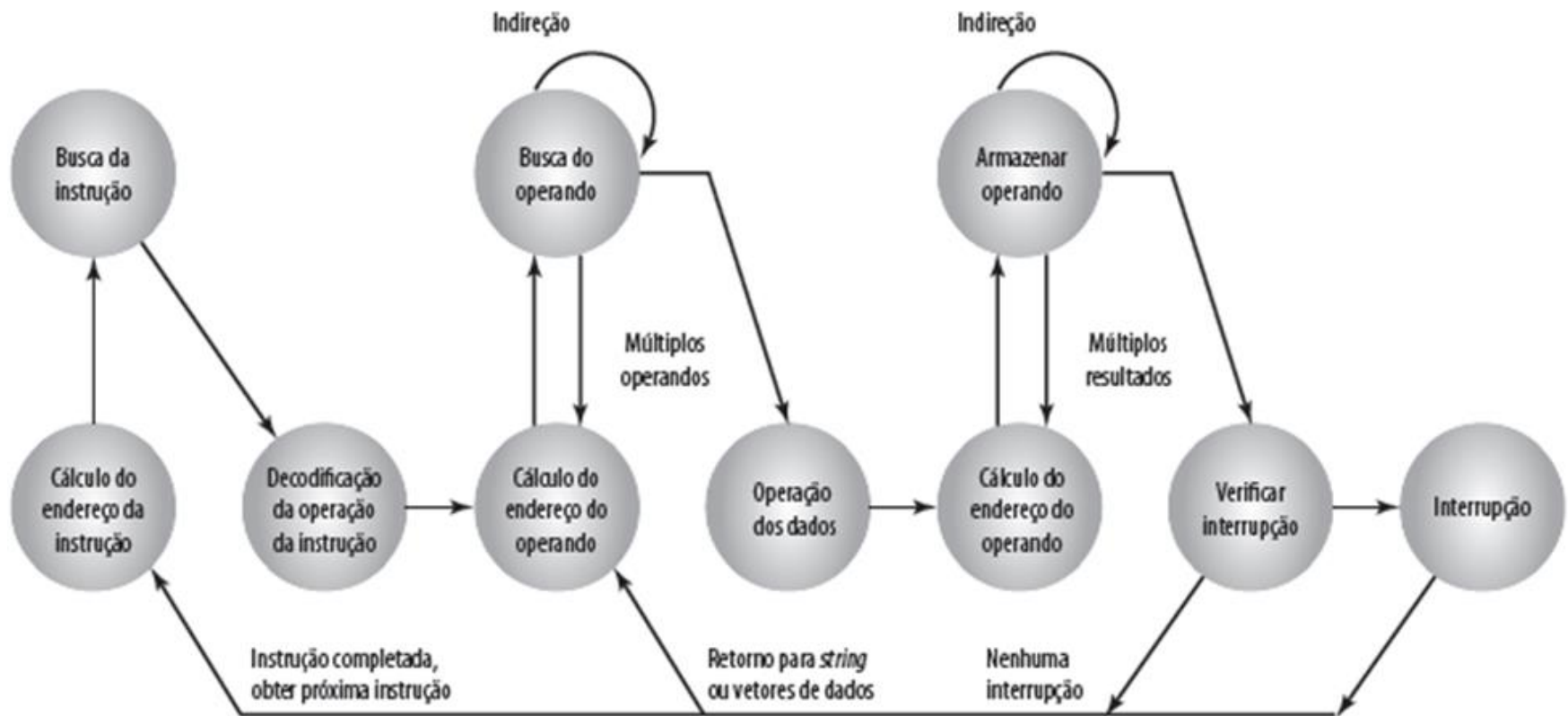
Ciclo de instrução (com interrupção)



Ciclo indireto

- Uma instrução pode exigir acesso à memória para obter operandos.
- Porém, endereçamento indireto requer mais acessos à memória (Lembrar de $EA = (A)$).
- Isto obriga o ciclo de instrução a realizar um “passo” a mais.

Diagrama de estado do ciclo de instrução



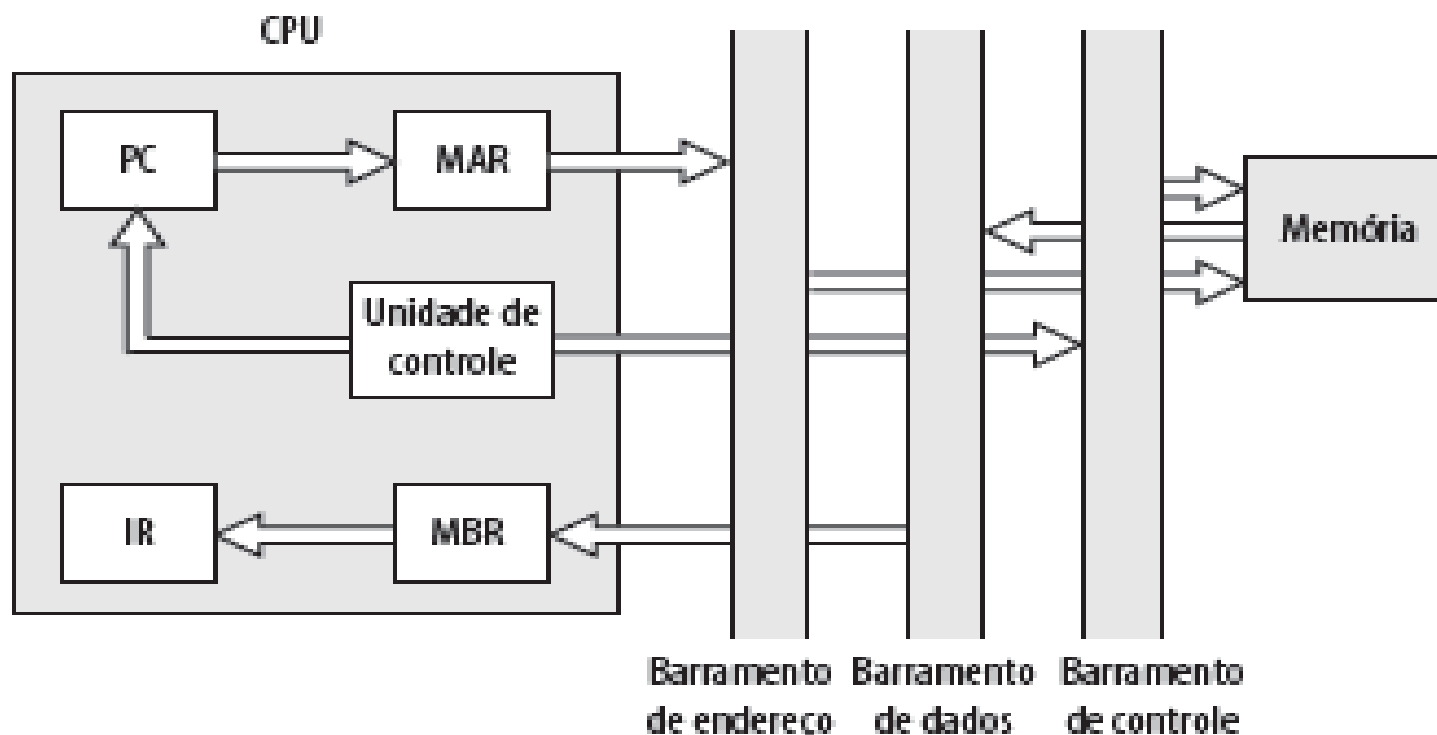
Busca de instruções: Fluxo de dados

- Depende do projeto da CPU.
- Em geral, a busca de uma instrução (*fetch*):
 - PC contém endereço da próxima instrução.
 - O endereço é carregado no MAR;
 - O endereço é disponibilizado no barramento de endereços;
 - A unidade de controle solicita uma operação de leitura da memória;
 - O resultado é trazido pelo barramento de dados, copiado no MBR e então para o IR;
 - O PC é incrementado.

Busca de instruções: Fluxo de dados

- IR é examinado.
- Se o endereçamento é indireto, o “ciclo indireto é realizado”:
 - Os N bits menos significativos do MBR são transferidos para o MAR (Quais N bits ?);
 - A unidade de controle requisita uma operação de leitura na memória;
 - O resultado (isto é, o endereço do operando) é movido para o MBR.

Busca de instruções: Fluxo de dados



MBR = registrador de buffer de memória

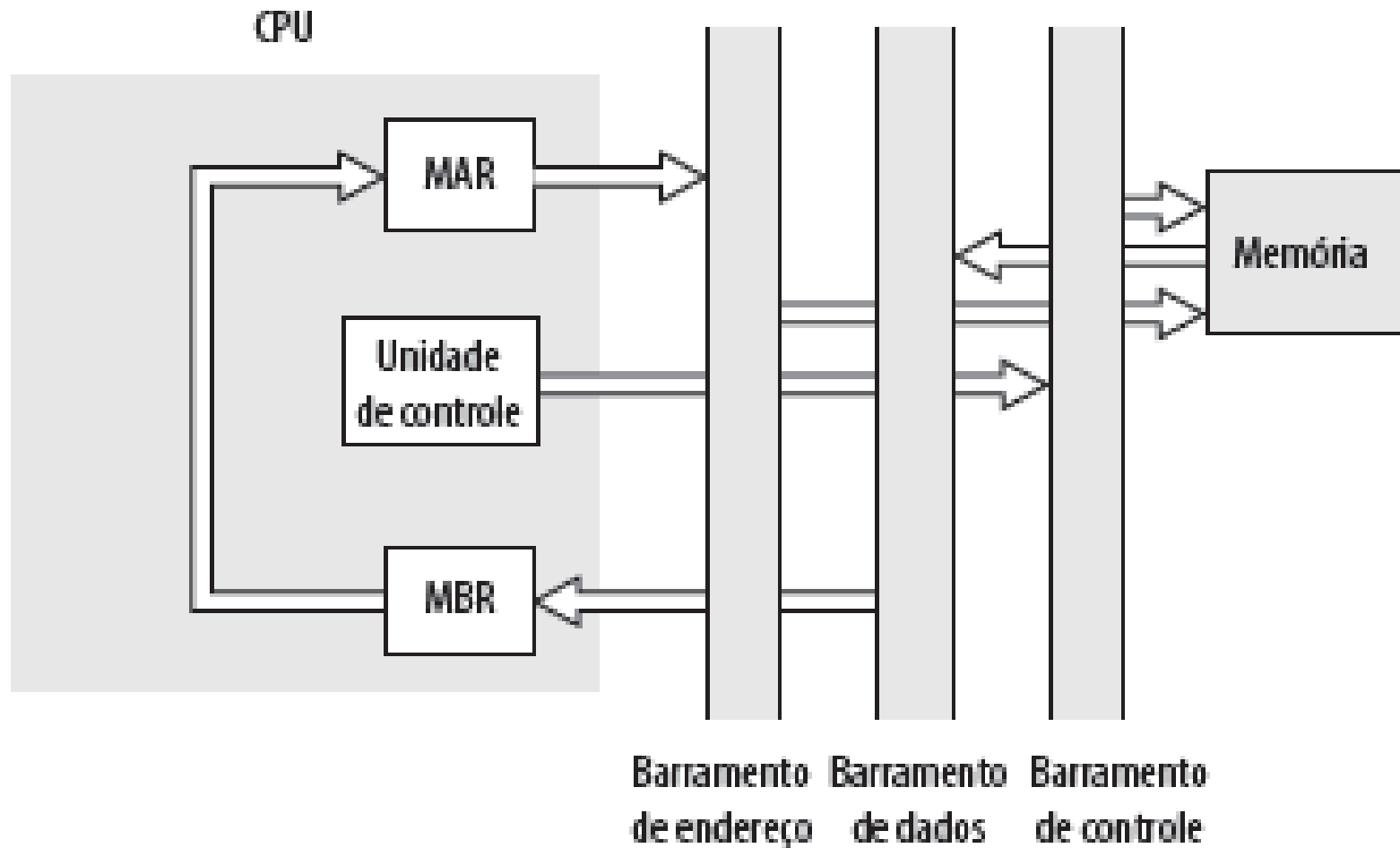
MAR = registrador de endereço de memória

IR = registrador da instrução

PC = contador de programa

Busca de instruções: Fluxo de dados

Ciclo Indireto



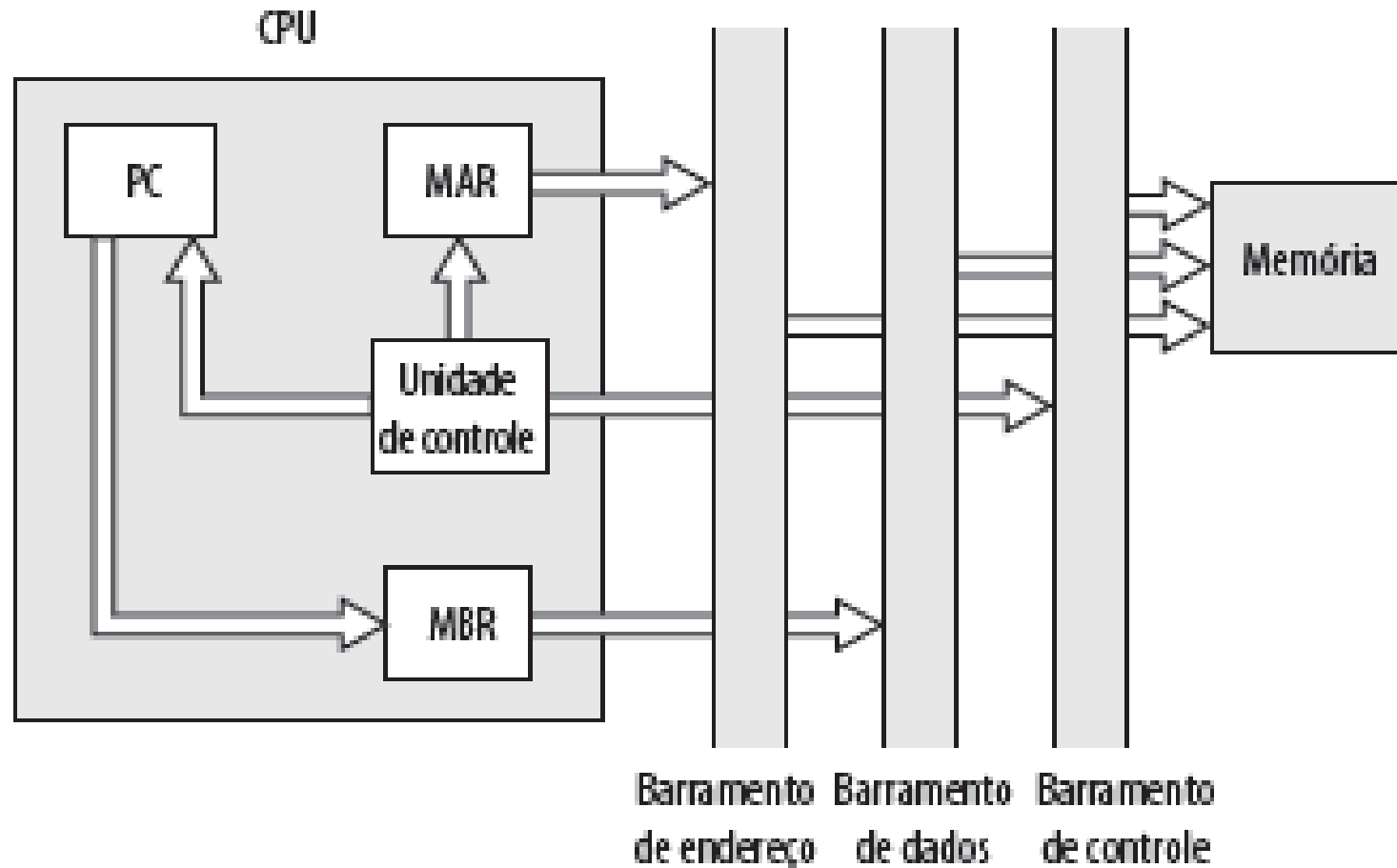
Fluxo de dados: Execução de instrução

- Os ciclos de instruções podem assumir várias formas.
- Depende da instrução em execução.
- Pode incluir:
 - Operação de leitura ou escrita na memória;
 - Operação de Input/Output;
 - Transferências entre registradores;
 - Operações com a ALU.

Fluxo de dados: Interrupção

- São simples e previsíveis:
 - O conteúdo do PC é salvo para permitir o retorno ao ponto anterior de execução – É copiado para o MBR;
 - O ponto de salvamento é carregado no MAR (Ex. *stack pointer*);
 - O MBR é escrito na memória;
 - O PC é carregado com o endereço da rotina de manipulação de interrupções;
 - A próxima instrução, a primeira do manipulador de interrupções, pode ser carregada.

Fluxo de dados: Diagrama de Interrupção



Busca Antecipada (*prefetch*)

- A busca da próxima instrução acessa a memória principal.
- A execução normalmente não acessa memória principal.
- Portanto, pode-se buscar a próxima instrução durante execução da instrução atual.
- Isto é chamado busca antecipada da instrução ou *instruction prefetch*.

O *prefetch* aumenta a performance?

- Aumenta a performance, mas não a dobra:
 - A operação de busca (*fetch*) é em geral mais curta que a execução
 - Buscar mais de uma instrução?;
 - Em casos de “jump” ou “branch” as instruções pré-buscadas (*prefetched*) não serão as instruções necessárias.
- Pode-se adicionar mais estágios para melhorar a performance.
 - **Pipeline!**

Bibliografia

- Sttallings, W. **Arquitetura e organização de computadores: projeto para o desempenho**. 8. ed. Prentice Hall, 2009.
- Delgado, J.; Ribeiro, C. **Arquitetura de Computadores**. 2 ed. LTC, 2009.
- Patterson, D. A. ; Hennessy, J.L. **Organização e projeto de computadores – a interface hardware software**. 3. ed. Editora Campus, 2005.